Cenni di Algoritmi e Strutture Dati

Lezione 1 - Algoritmi e BigO notation

Corso PON "Coding & App" - Istituto Superiore di Istruzione "Archimede", Treviglio

Cos’è un algoritmo?

È una descrizione ad alto livello dei passi di una procedura per la risoluzione di un insieme di problemi. La sua implementazione può essere eseguita su un computer fisico, utilizzando *memoria* e impiegando del *tempo*.

È necessario, dunque:

- Capire il problema che si vuole risolvere;
- Imparare a stimare l’efficienza computazionale di un algoritmo in termini di tempo e di memoria;
- Studiare tecniche algoritmiche basiche e strutture dati differenti per risolvere problemi che condividono la stessa strategia risolutiva.

Problema Algoritmico: Numero di Fibonacci

Definizione: sequenza numerica F1, F2, ..., Fn,.. in cui ogni elemento è la somma dei due numeri precedenti

Per convenzione, F0 = 0 e

$$F_n = \begin{cases} 1 & \text{if } n = 1 \text{ or } n = 2 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases}$$

Problema Algoritmico: Numero di Fibonacci

Esiste una formula ben definita per calcolare il numero di Fibonacci F_n:

$$F_n = \frac{1}{\sqrt{5}}\left(\phi^n - \hat{\phi}^n\right)$$

Eppure

Non è possibile calcolare Fn su una macchina data l'irrazionalità dei due caratteri

$$\phi^n - \hat{\phi}^n$$

Sono necessarie, quindi, altre soluzioni

Problema Algoritmico: Numero di Fibonacci

soluzione ricorsiva

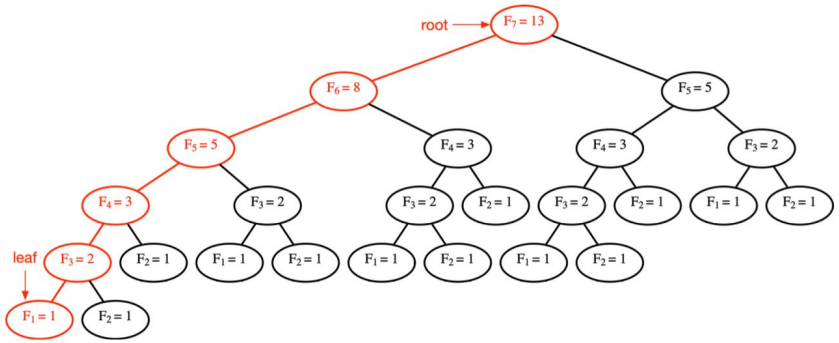
è possibile convertire la seguente formula matematica:

$$F_n = \begin{cases} 1 & n \leq 2 \\ F_{n-1} + F_{n-2} & n > 2 \end{cases}$$

nel seguente codice (esempio scritto in Python), utilizzando un algoritmo ricorsivo:

```python
def recursiveFib(n):
    if n <= 2:
        return 1
    return recursiveFib(n - 1) + recursiveFib(n - 2)
```

Numero di Fibonacci - albero ricorsivo



Le chiamate ricorsive sono gestite dallo stack. Ad un qualsiasi tempo T l'utilizzo di memoria dipende da tutte le chiamate attive al tempo T.

Il path massimo di chiamate ricorsive è legato al path leaf-root più lungo dell'albero di ricorsione.

CALCOLO TEMPO ESECUTIVO:

- **Secondi**? Troppo dipendente dalla macchina su cui l'algoritmo viene eseguito
- **Contare il numero di istruzioni**? Troppo dipendente dalla macchina su cui l'algoritmo viene eseguito

Contare il numero delle operazioni elementari

Tempo di esecuzione stimato - algoritmo ricorsivo

```
function FIB2(INT n) → INT
    if n ≤ 2 then
        return 1
    else
        return FIB2(n − 1) + FIB2(n − 2)
```

da cui possiamo estrarre la seguente relazione:

$$T(n) = \begin{cases} 1 & n \leq 2 \\ T(n-1) + T(n-2) + 1 & n > 2 \end{cases}$$

Tempo di esecuzione stimato - algoritmo ricorsivo

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&\geq 2T(n-2) + 1 && \text{Since } T(n-1) \geq T(n-2) \\
&\geq 4T(n-4) + 2 + 1 && \text{Since } T(n-2) \geq 2T(n-4)+1 \\
&\geq 8T(n-6) + 4 + 2 + 1 && \text{Since } T(n-4) \geq 2T(n-6)+1 \\
&\geq \ldots \\
&\geq 2^k T(n-2k) + \sum_{i=0}^{k-1} 2^i && \text{Generalization at the k-th step} \\
&\geq \ldots && \text{Stop when } n-2k = 0 \Rightarrow k = \lfloor n/2 \rfloor \\
&\geq 2^{\lfloor n/2 \rfloor} + \frac{2^{\lfloor n/2 \rfloor} - 1}{2-1} \\
&\geq 2^{\lfloor n/2 \rfloor}
\end{aligned}
$$

Problema Algoritmico: Numero di Fibonacci

soluzione iterativa

Consideriamo il seguente codice Python

```python
def iterativeFib(n):
    L = [0, 1, 1]
    for i in range(3, n + 1):
        L.append(L[i - 1] + L[i - 2])
    return L[-1]
```

a) creazione nuova lista ➡ **creazione**
b) Assegnazione L[0], L[1] e L[2] ➡ **assegnamento**
c) iterazione (n-2) per l'assegnamento, per la somma tra (n-1) e l'assegnamento ad L

Total: $4 + 3(n-2) = 3n - 2$ elementary operations (for $n > 2$)

Problema Algoritmico: Numero di Fibonacci

soluzione con calcolo matriciale

Consideriamo il seguente teorema, dimostrabile per induzione

Consider $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. For every $n \geq 2$ we have that

$$A^{n-1} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} = \begin{pmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{pmatrix}$$

Problema Algoritmico: Numero di Fibonacci

soluzione con calcolo matriciale

Considerazioni aggiuntive:

- se n è pari, allora:

$$A^n = (A^{n/2})^2, A^{n+1} = (A^{n/2})^2 \cdot A$$

è possibile scrivere un codice con tempo proporzionale a log(n), in quanto è possibile richiamare un numero log(n) chiamate ricorsive in modo da calcolare

$$A^n$$

Ma cosa vuol dire

“tempo proporzionale a log(n)”?

Cost function - definizione

Dato un qualsiasi **n ≥** 0, definiamo **f(n) ≥** 0 l'ammontare di risorse (in termini di tempo e di memoria) necessario per l'esecuzione di un generico algoritmo, dato un input di dimensione **n**

<u>In generale</u>

siamo interessati a valutare il tasso di crescita della *cost function f(n).*

<u>In particolare</u>

si va a considerare il comportamento asintotico della *cost function f(n)*

<u>in modo da rispondere alla domanda:</u>

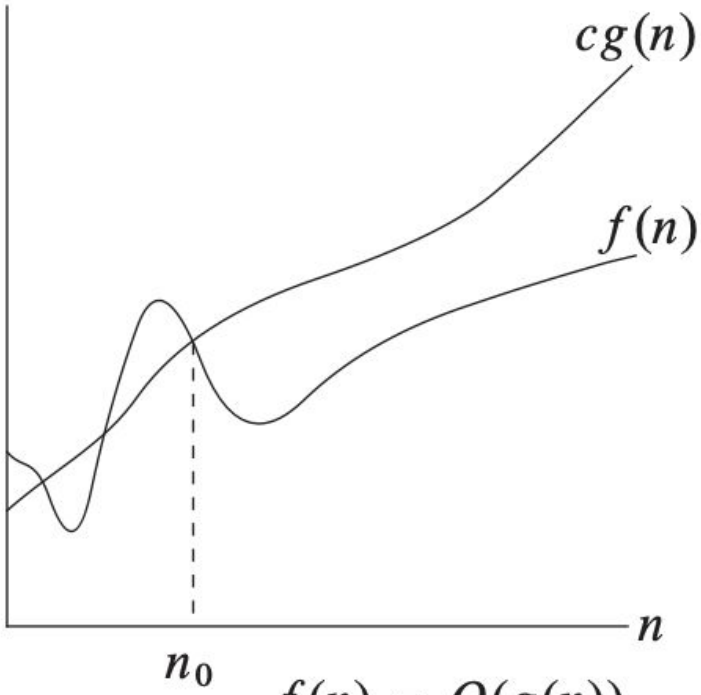
"quale algoritmo ha performance migliori in termini di ***tempo di esecuzione?****"*

Big-O notation - definizione

Data una cost function g(n), si definisce un insieme per il quale g(n) è un limite superiore asintotico come:

$$O(g(n)) = \{f(n) \mid \exists c > 0, n_0 \geq 0 \text{ such that } \forall n \geq n_0, f(n) \leq cg(n)\}$$

$$O(g(n)) = \{f(n) \mid \exists c > 0, n_0 \geq 0 \text{ such that } \forall n \geq n_0, f(n) \leq cg(n)\}$$



$f(n) = O(g(n))$

Applicazione definizione Big-O

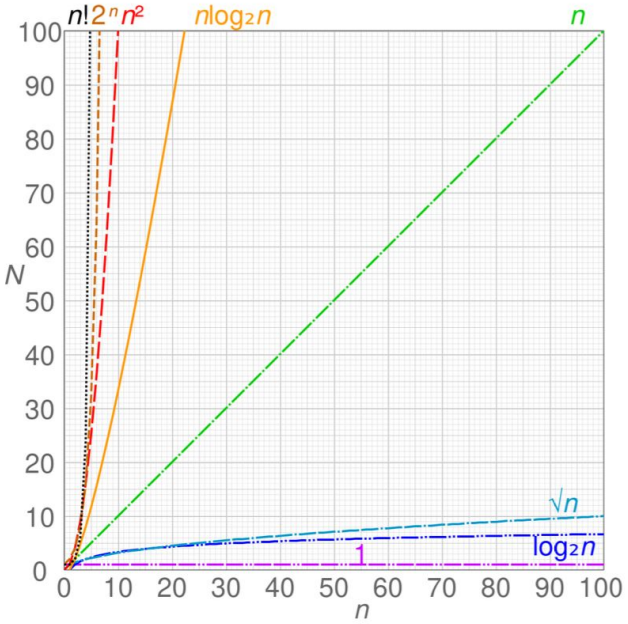
Generalmente, Big-O non viene applicato in quanto definizione, dato che il comportamento asintotico è ben definito e, a livello di complessità algoritmica, si tende ad approssimare molto.

<u>È possibile, tuttavia, utilizzarla.</u>

Sono presenti altri tre indicatori, definiti come Big-Omega, Little-omega Theta, e Little-o, per identificare, ad esempio, comportamenti asintotici con tasso di crescita maggiore (Big-Omega), con tasso di crescita uguale (Theta) o con tasso di crescita minore (Little-o).

In questo corso verranno, tuttavia, utilizzati solo come notazione.

Order of Growth	Name
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(\log^k n)$	Polylogarithmic, $k \geq 1$
$O(n^k)$	Sublinear, $0 < k < 1$
$O(n)$	Linear
$O(n \log n)$	Pseudolinear
$O(n^k)$	Polynomial, $k > 1$
$O(n^2)$	Quadratic, for $k = 2$
$O(n^3)$	Cubic, for $k = 3$
$O(c^n)$	Exponential, base $c > 1$
$O(n!)$	Factorial
$O(n^n)$	Exponential, base n



Analisi di Best, Average, Worst case scenarios

Best Case

descrive il comportamento algoritmico sotto condizioni ottimali

es. ricerca lineare, dove l'elemento da cercare è il primo

Worst Case

descrive il comportamento algoritmico sotto condizioni non favorevoli

es. ricerca lineare, dove l'elemento da cercare è l'ultimo

Average Case

descrive il comportamento algoritmico medio su tutti gli input possibili

es. tempo medio necessario per la ricerca lineare